COMP20005 Workshop Week 5

functions

1 Functions. Discuss: Ex. 5.01 5.02, 5.03

2 start-together: 5.05 or 5.06
implement 5.05, 5.06

3

```
while ( having_time() ){
    do("a new exercise in Chapter 5");
}
```

Library Functions

How to:

print out the value $\sqrt{2}$.

(Library) functions are black boxes

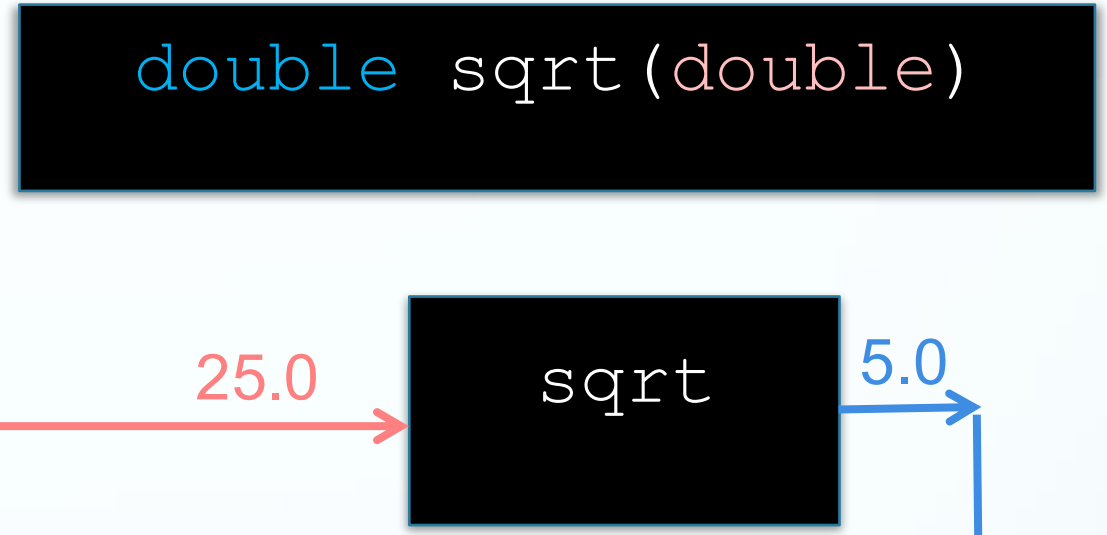
my_prog.c

```
#include <stdio.h>
#include <math.h>

int main(...) {
  double x=3.0, z;

  z= sqrt( x*x + 4*4 );

  printf("z= %f\n", z);
  return 0;
}
```



Last Week's Lectures

- A function is a useful abstraction.
- We can, and should, design and implement our own custom functions!

Why functions?

- Avoid Repetition.
- Build bigger programs from smaller ones
- Think abstractly. Hence:
  - "Think big" when designing a program
  - Focus on logic, not messy details

Last week solution for finding the next prime

```
... scanf("%d", &n) ...
while (1) {
  n++;
```

input:
n

```
int isprime = 1, div;
for (div = 2; div * div <= n; div++) {
  if (n % div == 0) {
    isprime = 0;
    break;
  }
}
// - if isprime == 1 ➔ prime
// - if isprime == 0 ➔ not prime
```

output:
isprime

```
  if (isprime) {
    break;
  }
}
printf("next prime= %d\n", n);
```

Functions - Useful Abstraction

Exercise 4.09 revisited:

Write a program nextprime that calculates the *next* prime number after a given value.

Alistair's code for checking if n is a prime number

Last week solution

```
... scanf("%d", &n) ...
while (1) {
   n++;
```

input: **n**

```
int isprime = 1, div;
for (div = 2; div * div <= n; ...
   if (n % div == 0) {
      isprime = 0;
      break;
   }
}
// - if isprime == 1 ➔ prime
// - if isprime == 0 ➔ not prime
```

output: **isprime**

```
  if (isprime) {
     break;
  }
}
printf("next prime= %d\n", n);
```

Using a Function as an Abstraction

```
      ... scanf("%d", &n) ...
      while (1) {
         n++;
         if (is_prime_number(n)) {
            break;
         }
      }
      printf("...", n);
      return 0;
   }

int is_prime_number(int n) {
```

```
      int isprime = 1, div;
      for (div = 2; div * div <= n; div++){
         if (n % div == 0) {
            isprime = 0;
            break;
         }
      }
```

```
   return isprime;
}
```

value 0(false) or 1(true)

Anatomy of a C Function

Structure: Each function has:

- A **name**
- Inputs (parameters)
- A process (the body)
- An output (return value)

```
return_type function_name(parameter_list) {
   // function body
   // statements to be executed
   return value; // if return_type is not void
 }
```

Key Components:

- **return_type**: The type of data the function sends back after it's done (e.g., int, float, void).
- **function_name**: A unique name that follows variable naming conventions.
- **parameter_list**: The list of variables the function takes as input. This can be empty.
- **function body**: The code block that contains the instructions to perform the task.
- **return statement**: Passes the result back to the calling function.

pass-by-value: parameters are always passed by values

When executing function call

add(a, b+2):

- the value of *expression* a+4 is computed and passed to m
- the value of *expressions* b is computed and passed to n

when the function finishes, the value of *expression* total is computed and sent back to the caller

```
int main(...) {
  int a=2, b=4, sum= 0;

  sum = add(a+4, b );

  // sum= ?
  // n = ?
  // b = ?

 ...
}
```

6 is passed to m

4 is passed to n

```
int add(int m,    int n) { // m and n are local in add
  int total= m + n;
  n= 20;
  return total;
}
```

10

- Through parameter n, function add receives a copy of the argument's value. Consequently, it cannot modify the original variable (e.g., b) in the calling function.

- Via return, a function sends a value back to the caller.

	my_program.c

#include & #define
function prototypes
 	
main() function –start
 	
-end
	#include <stdio.h>
	int add(int m, int n);
	
	int main(int argc, char *argv[]) {
	
	int a=2, b=4;
	printf("sum of a+4 and b is %d\n",
	add(a+4, b);
	return 0;
	}

implementation of
declared functions
	int add(int m, int n) {
	int total= m + n;
	return total;
	}

Discuss and Do-Together Ex 5.01, 5.02, 5.06

Ex. 5.01 (largest of 2 ints) and 5.02 (largest of 4 ints):

- You only need to write a function, not the main() function. This is exactly the requirement for the programming question in the upcoming MST.

Ex. 5.x1, 5.03 onwards:

- Your main task is to write a function.
- However, you also need to provide a main() function that tests the function you are asked to write (in an exam/test setting, you are not supposed to do this unless explicitly instructed).
- For today, if you don't have enough time, consider just to write the required function, and add the main() when having time.

Lab

- Finish 5.01, 5.02, 5.03 if not yet done (ask friends/Anh for help)
- Complete 5.05 and 5.06 (as a minimum requirement)
- Extras: improve 3.06 using functions, 5.x1, 5.06b, 5.11
- If you are confident: 5.08, 5.13, 5.14, 5.08b, 5.15

For each task, spend a few minutes *before writing* to discuss with your peers:

- WHAT are possible approach(es)?
- WHICH APROACH is simpler for us to write?

- MST:
  - Review for MST & Ask Questions

Remember

Review chapters 1-5 for the MST:

- program structures, data types, expression, precedence order
- scanf and printf
- if ...
- loops: for ... , while ...
- functions, recursive functions
- better not to use: switch ... , do ... while

At home: Preparing for the Mid-Semester Quiz on Thursday Week 6 by

- Reviewing lectures
- Practicing programming using exercises in Chapter 1 – Chapter 5, especially writing functions.
- Trying the practice test with timing. Note: you can do a few times, and you'll probably have different questions each time.
- Making sure about the MST: Where, When, What Rules

Additional Slides

Recursive functions [time permits]

Recursive function: function that calls itself

```
int fib( int n ) {
  if (n==1) return 1;
  if (n==2) return 1;
  return fib(n-1) + fib(n-2);
}
```

```
int factorial( int n ) {
  if (n==1) {
    return 1;
  }
  return n * factorial(n-1);
}
```

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) { //n=3
  if (3<=1) return 1;

  return 3*f(2);
}
```

```
int f(int n) { //n=2
  if (2<=1) return 1;

  return 2*f(1);
}
```

```
int f(int n) { //n=1
  if (1<=1) return 1;

  return n*f(n-1);
}
```

```
int f( int n ) {
  if (n<=1) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3) ;
  // k is 6
}
```

```
int f(int n) {
  if (2<=1) return 1;

  return 3* f(2) ;
}
```

6

```
int f(int n) {
  if (2<=1) return 1;

  return 2* f(1) ;
}
```

2

```
int f(int n) {
  if (1<=1) return 1;

  return n*f(n-1);
}
```

1

```
int f( int n ) {
  if (n) {
    return 1;
  }
  return n*factotial(n-1);
}
```

Recursive functions: How

- reduce the task of size n to the same tasks of smaller sizes
- clearly describe the base cases where the solutions are trivial
- when writing code, start with solving the base cases first

Examples: factorial (n):

- base case: when n==1 the solution is 1
- general case: factorial(n) can be computed from factorial(n-1)

```
int factorial( int n ) {
  if (n==1) { // base case
    return 1;
  }
  // general case    [note: else is normally not needed]
  return   factorial(n-1)*n;
}
```

- Do-Together **Exercise 5.13**